Results in Engineering 29 (2026) 109857

Contents lists available at ScienceDirect

Results in Engineering

journal homepage: www.sciencedirect.com/journal/results-in-engineering

Research paper

Agentic AI home energy management system: A large language model framework for residential load scheduling

Reda El Makroum [a,*], Sebastian Zwickl-Bernhard [a,b], Lukas Kranzl [a]

[a] Energy Economics Group (EEG), Technische Universität Wien, Gußhausstraße 25–29/E370-3, 1040, Vienna, Austria
[b] Department of Industrial Economics and Technology Management, The Norwegian University of Science and Technology, Trondheim, Norway

ARTICLE INFO

Keywords:
Agentic AI
Home energy management systems
Large language models
Demand response
Load scheduling

ABSTRACT

The electricity sector transition requires increased residential demand response capacity, yet Home Energy Management Systems (HEMS) adoption remains limited by user interaction barriers. Large language models (LLMs) offer potential to address these barriers through natural language interaction; however, existing implementations employ LLMs as rule generators, optimizers, or parameter extractors rather than autonomous coordinators for multi-appliance scheduling. This paper presents an agentic AI HEMS where LLMs autonomously coordinate scheduling from natural language requests. A hierarchical architecture combining one orchestrator agent with three specialist agents is developed using the ReAct pattern, enabling dynamic coordination. Evaluation across three open-source models (Llama-3.3-70B, Qwen-3-32B, GPT-OSS-120B) using real Austrian day-ahead electricity prices reveals substantial capability differences for multi-appliance coordination. Llama-3.3-70B successfully coordinates all three appliances across all evaluation scenarios to match cost-optimal benchmarks computed via mixed-integer linear programming, while Qwen-3-32B and GPT-OSS-120B struggle to coordinate all appliances simultaneously despite achieving perfect single-appliance performance. The demonstrated system enables natural-language-based scheduling without technical parameter specification, offering a pathway to address configuration complexity barriers that currently limit residential HEMS adoption. All system components, including complete agent prompts, orchestration logic, and simulation user interfaces, are released as open source to enable reproducibility and further development.

1. Introduction

1.1. Setting the stage

The global energy transition requires fundamental changes in how electricity is generated, distributed, and consumed across all sectors. According to the International Energy Agency, electricity grids must integrate growing shares of variable renewable generation while maintaining reliability and affordability, creating unprecedented demand for system flexibility [1]. This transformation affects all sectors, with flexibility resources needed to balance supply and demand in real time. The agency further projects that global demand response capacity must reach 500 GW by 2030, representing a tenfold increase from 2020 levels, with buildings and residential electric vehicles (EVs) accounting for approximately 60% of this potential [2]. The residential sector thus emerges as a critical resource due to its scale and diversity of controllable loads. These flexible residential loads can shift consumption across time without compromising user requirements, representing significant potential for demand response and renewable energy integration [3].

Realizing this potential requires systems that coordinate load scheduling to minimize electricity costs while respecting user constraints. Home Energy Management Systems (HEMS) address this need through software platforms that optimize household energy consumption via intelligent scheduling and control of appliances and distributed energy resources [4]. These systems have demonstrated technical effectiveness in reducing electricity costs and enabling demand response participation [5]. However, widespread residential adoption remains limited. The IEA projects that meeting global decarbonization targets requires HEMS deployments to increase from 4 million units in 2020 to 32.7 million units by 2030, yet current adoption trajectories fall far short of this eightfold growth requirement [2]. While financial barriers and system complexity contribute to this gap [6], user interaction challenges emerge as a critical bottleneck [7]. Existing HEMS interfaces require users to translate everyday preferences into numerous well-formatted technical parameters, interpret often-confusing parameter documentation, and provide precise inputs for optimization algorithms. This process is time-consuming and discourages adoption among non-expert users, particularly the elderly and those with limited

* Corresponding author.
E-mail addresses: elmakroum@eeg.tuwien.ac.at (R. El Makroum), zwickl@eeg.tuwien.ac.at (S. Zwickl-Bernhard), kranzl@eeg.tuwien.ac.at (L. Kranzl).

https://doi.org/10.1016/j.rineng.2026.109857
Received 18 December 2025; Received in revised form 10 February 2026; Accepted 1 March 2026
Available online 3 March 2026

technical literacy, thus diminishing HEMS democratization through steep learning curves [8]. Addressing this interaction barrier through intuitive, conversational interfaces represents a key enabler for widespread HEMS deployment. However, simply augmenting existing HEMS with a natural language interface does not fully address this barrier. While such an approach improves input accessibility, the system remains limited to the scheduling functions the underlying optimizer was designed for. A conversational agentic approach enables capabilities that extend beyond scheduling, including answering household energy queries, explaining scheduling decisions, identifying consumption inefficiencies, and proactively recommending behavioral adjustments. These advisory and analytical functions require reasoning capabilities that optimization backends cannot provide regardless of their interface layer.

Natural language interfaces offer promise for addressing these interaction challenges. Recent advances in large language model (LLM) capabilities, particularly in tool calling and multi-step reasoning, combined with emerging agentic AI frameworks have enabled new approaches to system automation. LLMs can operate without example demonstrations [9,10], relying on pre-trained capabilities and task descriptions, which is particularly valuable for residential HEMS where users cannot be expected to provide example scheduling requests. When structured as agentic systems, LLMs can autonomously decompose tasks, invoke tools, and coordinate multi-step processes through hierarchical architectures [11]. Such systems could simplify HEMS interaction and reduce cognitive complexity by allowing users to express scheduling preferences conversationally while the system handles technical translation and device control. Despite these capabilities, no existing HEMS implementation deploys an agentic AI system where LLMs autonomously manage the full workflow from natural language input to device scheduling. Given this gap, we formulate the following research question: How can a reliable, secure, and effective agentic AI system for home energy management systems be designed, developed, and simulated? To position this research, we review how agentic AI systems are conceptualized in recent literature and how LLMs have been applied to energy research.

1.2. Related work

We first examine how agentic AI is defined and characterized in recent literature, then review existing efforts to integrate LLMs into energy system applications. The literature landscape reflects the emerging state of this field, with most relevant work emerging in 2024–2025. Sapkota et al. [12] distinguish between task-specific AI agents and agentic AI systems, defining the latter as systems that use LLMs as autonomous reasoning engines to coordinate multiple specialized agents and dynamically decompose complex objectives into executable sub-tasks without predefined workflows. Hosseini and Seilani [13] complement this by identifying hierarchical reasoning and adaptability as essential capabilities separating agentic systems from traditional automation. Together, these perspectives establish that agentic AI systems combine multi-agent coordination through LLMs with adaptive capabilities, distinguishing them from single-purpose LLM applications.

Having reviewed how agentic AI is conceptualized in recent literature, we now examine how LLMs have been applied to various energy system challenges. Majumder et al. [14] identify that effective LLM deployment in the energy sector requires domain-specific adaptations including fine-tuning and retrieval-augmented generation (RAG). Demonstrating these in practice, Zhang et al. [15] integrate LLMs into demand side management for electric vehicle charging, employing retrieval-augmented generation for automatic problem formulation and code generation. They demonstrate effectiveness in charging scheduling optimization, but focus on generating optimization code rather than autonomous agentic coordination.

Looking at the buildings sector, Shu and Zhao [16] evaluate seven LLMs on residential retrofit decisions using 400 homes across 49 US states. They find that LLMs achieve up to 92.8% top-5 recommendation accuracy but demonstrate simplified reasoning that lacks deeper contextual awareness of local economic and behavioral factors. Eshbaugh et al. [17] develop a multimodal generative AI framework for synthetic building energy data generation, addressing data accessibility constraints. Chen et al. [18] develop a customized LLM for building energy system operation and maintenance using few-shot learning for task routing and retrieval-augmented generation for domain knowledge integration. They achieve 96.3% accuracy for fault diagnosis and outperform general LLMs, but focus on diagnostic recommendations rather than autonomous device control and scheduling. Sawada et al. [19] deploy Office-in-the-Loop, an agentic AI system for HVAC control in operational office environments, integrating sensor data and occupant feedback. They demonstrate 47.92% energy savings and 26.36% comfort improvements through real-world deployment, but focus on single-system control in commercial buildings rather than residential multi-appliance coordination.

Closing in on LLM applications for home energy management, Giudici et al. [20] apply GPT models to generate HomeAssistant automation routines from natural language commands, demonstrating proficiency in JSON generation and user engagement in empirical evaluation. Their system generates automation rules from user requests but does not provide autonomous multi-agent coordination or dynamic scheduling optimization. Li et al. [21] integrate AI agents, LLMs, and Digital Twin technology into smart home energy management, achieving 47.77% energy reduction through context-aware decision-making. While their system demonstrates real-time optimization capabilities, it focuses on general energy-saving actions rather than coordinated scheduling of specific residential appliances through natural language interaction. Raghavan and Giridhar [22] propose an LLM-MPC framework where a zero-shot LLM (Phi-3 Mini) interprets natural language commands and orchestrates tool calls to an underlying Model Predictive Control system for battery scheduling optimization, demonstrating improved financial outcomes compared to baseline approaches. Gkalinikis et al. [23] present RHEA, a residential home energy advisor providing personalized energy recommendations through a conversational interface. Most closely related to the system presented in the paper is a LLM-based interface for HEMS developed by Michelon et al. [8] that extracts user preferences and schedules from natural language dialogue, achieving 88% parameter retrieval accuracy using ReAct for multi-turn interactions. Their approach positions LLMs as parameterization interfaces for traditional optimization-based HEMS, addressing the user interaction challenge but maintaining the separation between natural language processing and energy management logic.

These existing efforts primarily employ LLMs as preprocessing or interface tools that feed traditional decision-making systems, rather than as autonomous decision-making entities. Table 1 summarizes how the present work differs from existing LLM-based energy applications across key architectural dimensions.

1.3. Key contribution

Building on the identified gap in existing literature, this paper introduces the following three contributions:

- In contrast to existing work that uses LLMs as preprocessing tools (generating code, extracting parameters, making recommendations), we present a home energy management system where the LLM serves as the autonomous decision-making entity, employing iterative ReAct-based reasoning to coordinate hierarchical multi-agent workflows and directly executing scheduling decisions without intermediate handoff to conventional optimization systems. We open-source the complete system including orchestration logic, agent prompts, tools, and web interfaces to enable reproducibility, extension, and future research.
- We introduce a hierarchical multi-agent architecture combining one orchestrator agent with three specialist agents that employ iterative

Table 1
Summary of LLM-based energy system applications, outlining their key architectural distinctions.

Ref	Approach	LLM Role	Output	Key Distinction
Zhang et al. [15]	EV charging DSM with RAG	Generates optimization code for external solver	Charging schedules via solver execution	LLM as code generator; no direct control or NL interaction.
Chen et al. [18]	Building O&M with few-shot learning	Diagnoses faults and routes tasks	Diagnostic recommendations	Single-agent; recommendations only, no device actuation.
Sawada et al. [19]	Commercial HVAC control	Controls single HVAC system with sensor feedback	Setpoint adjustments	Real-world deployment; single-system focus, commercial context.
Giudici et al. [20]	HomeAssistant automation	Generates JSON automation rules from NL	Static automation routines	Rule generation only; no dynamic scheduling or optimization.
Li et al. [21]	Smart home with Digital Twin	Optimizes general energy actions	Energy-saving recommendations	Digital Twin integration; no NL input or appliance-specific scheduling.
Gkalinikis et al. [23]	Residential energy advisory	Provides personalized energy recommendations	Energy advice via conversational interface	Advisory focus; no scheduling optimization or device control.
Raghavan and Giridhar [22]	Residential HEMS with LLM-MPC	Interprets NL commands for MPC orchestration	Battery schedules via MPC solver	LLM as interface to MPC; single-system focus, no multi-appliance coordination.
Michelon et al. [8]	Residential HEMS parameterization	Extracts preferences via ReAct dialogue	Parameters for external optimizer	LLM as interface layer; scheduling delegated to conventional solver.
Present study	**Residential HEMS with agentic LLM-based coordination**	**Autonomous orchestration of specialist agents**	**Optimal appliance schedules**	**LLM as decision-maker; multi-agent architecture with NL input and device control.**

reasoning and acting cycles. In contrast to single-agent approaches in existing LLM-based energy applications, this enables multi-appliance coordination where specialized agents handle distinct load types (washing machine, dishwasher, EV charger).
- We develop a multi-layer tool framework that unifies analytical capabilities, external API integration, and specialist agent delegation within a single coordinated system, extending beyond the isolated scheduling or recommendation functions found in prior LLM-energy applications.

1.4. Paper structure

The remainder of this paper is structured as follows. Section 2 describes the hierarchical multi-agent architecture, ReAct-based orchestration, tool ecosystem design, and evaluation methodology. Section 3 presents system performance results across single-appliance, multi-appliance, and analytical query scenarios. Section 4 discusses deployment considerations including comparison to traditional approaches, model selection trade-offs, prompt and context engineering, security, and current limitations. Section 5 concludes with synthesis of key findings and research directions.

2. System design and implementation

The system coordinates multi-appliance energy scheduling through a hierarchical multi-agent architecture where one LLM-based orchestrator delegates tasks to three specialist agents, enabling autonomous coordination from natural language requests to device control while maintaining modular extensibility for diverse appliance types. Fig. 1 presents the complete architecture of the agentic AI HEMS, illustrating the multi-agent coordination framework, ReAct-based orchestration, specialist agent design, and API integration layer.

2.1. Agentic AI architecture

The system employs a hierarchical agentic AI architecture comprising one orchestrator agent and three specialist agents, all powered by the same LLM. This design separates strategic coordination from domain-specific optimization, enabling modular and extensible appliance scheduling. The architecture supports multiple LLM backends, with evaluation conducted across three models: Llama-3.3-70B [24], Qwen-3-32B [25], and GPT-OSS-120B [26]. All models are accessed through Cerebras' free-tier API (14,400 requests per day) with inference speeds of approximately 2500 tokens per second [27]. All agents operate with temperature set to 0.0[1] to achieve consistent scheduling decisions, though output variations may still occur due to floating-point precision and inference implementation details. Token efficiency strategies are detailed in D and Section 4.4

The orchestrator agent serves as the central coordinator, responsible for parsing user requests, fetching shared data resources (electricity prices, calendar events), delegating scheduling tasks to specialist agents, and executing final setpoints through the device control layer. The orchestrator uses iterative reasoning to adapt its coordination strategy to diverse user requests and system states.

Each specialist agent is an expert in scheduling a single appliance type. The three implemented specialists are:

- **Washing Machine (WM) agent:** Optimizes washing cycles by evaluating all valid time windows to minimize electricity cost. The agent performs sliding window analysis across 96 daily timeslots, calculating cumulative costs for each consecutive 8-slot window and selecting the minimum-cost option that satisfies user-defined deadline constraints.
- **Dishwasher (DW) agent:** Schedules dishwasher cycles using the same sliding window cost minimization approach. The agent evaluates all valid 6-slot windows and recommends the schedule with lowest total electricity cost while respecting deadline requirements.
- **EV charger agent:** Optimizes 6-h EV charging sessions with calendar-driven deadline awareness. Unlike the fixed-schedule appliances, the EV agent integrates constraints inferred from Google Calendar events, enabling the system to infer charging deadlines from calendar events based on user schedules without explicit instruction.

Each specialist agent receives tailored inputs from the orchestrator based on its scheduling requirements: all agents receive day-ahead electricity price data, while load-specific inputs such as calendar-extracted deadlines are provided to the EV agent. Each specialist agent has access to the *calculate_window_sums* tool as part of its toolset, enabling it to

[1] Temperature is an LLM inference parameter controlling output randomness, where 0.0 produces deterministic outputs by selecting the highest-probability token.

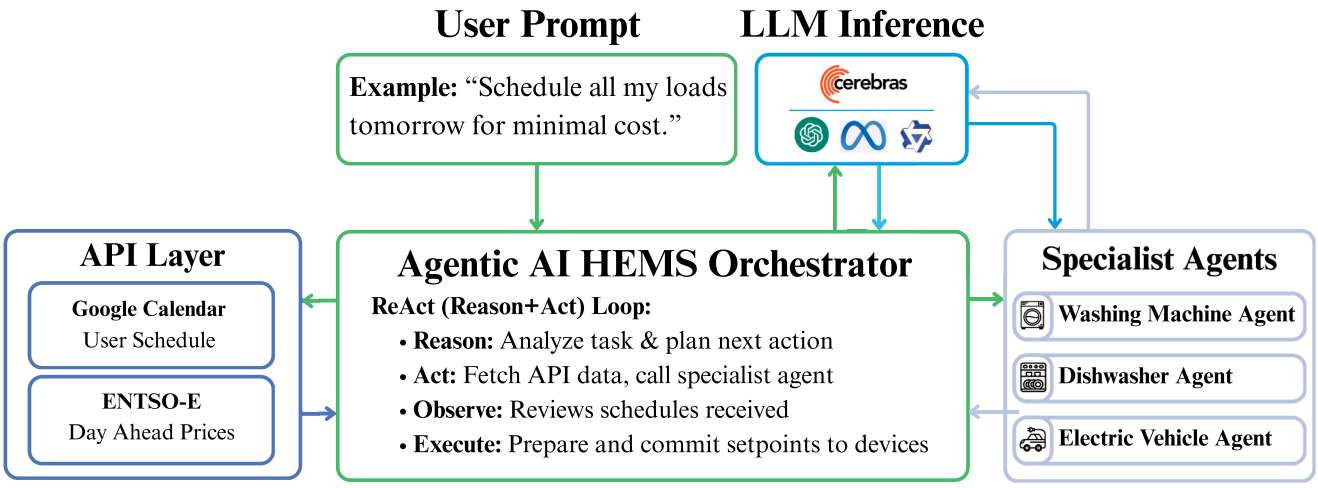

Fig. 1. Agentic AI HEMS Architecture: A central orchestrator agent coordinates three specialist load agents using the ReAct pattern, leveraging external APIs for price and calendar data, and committing optimized schedules to smart home devices.

evaluate all valid time windows and identify the minimum-cost schedule through exhaustive analysis. Unlike the orchestrator which employs iterative ReAct cycles, specialist agents operate in single-turn interactions, receiving inputs from the orchestrator, calling the window analysis function with appliance-specific duration parameters, processing the results using LLM reasoning, and formulating a structured recommendation including start slot, duration, estimated cost, and rationale in a single response. This single-turn design minimizes token consumption and reduces latency for domain-specific scheduling tasks. The orchestrator then aggregates these recommendations into schedule output files (format detailed in D). This hierarchical design ensures computational scalability: because each specialist agent optimizes independently, system complexity scales linearly with appliance count rather than combinatorially, enabling extension to larger device portfolios without architectural modification.

The system operates without example demonstrations, with neither the orchestrator nor specialist agents receiving example user requests, demonstrations of optimal schedules, or coordination workflow examples. Agent prompts contain only tool descriptions, task instructions, and operational guidelines (complete prompts are provided in Appendix C). The orchestrator receives descriptions of its six available tools but no demonstrations of how to sequence them for multi-appliance coordination. Specialist agents receive appliance specifications and price data but no examples of optimal scheduling decisions for given price patterns. This design enables the system to achieve optimal scheduling purely through LLM reasoning capabilities without few-shot learning or domain-specific fine-tuning, relying solely on tool descriptions and task instructions to coordinate multi-appliance energy management.

2.2. ReAct-based orchestration

The orchestrator agent employs the ReAct pattern to coordinate scheduling tasks through iterative decision-making [28]. This pattern is adopted to enable transparent multi-step reasoning across six orchestrator tools, providing explicit reasoning traces that facilitate debugging and system validation. Unlike specialist agents which operate in single-turn interactions for streamlined domain-specific optimization, the orchestrator requires flexible coordination of multiple tools in adaptive sequences determined by user requests and intermediate results. The ReAct pattern provides a systematic framework for the agent to reason about system state, decide on actions, execute them, observe outcomes, and adapt its strategy accordingly.

Each orchestration cycle follows an iterative loop where the agent alternates between reasoning and acting. In the reasoning phase, the agent analyzes the current state, user request, and available information to determine the next action. In the acting phase, the agent invokes one of six available tools:

- *get_electricity_prices:* Fetches day-ahead electricity prices from the ENTSO-E Transparency Platform [29] API. Returns 96 price points at 15-min resolution (00:00-23:45) in EUR/kWh, providing the foundation for cost optimization across all appliances.
- *get_calendar_ev_constraint:* Queries the Google Calendar API via OAuth to retrieve upcoming events within a specified time window. The orchestrator uses LLM reasoning to infer scheduling constraints from minimal event information (title and time only), enabling context-aware deadline extraction without explicit user instruction.
- *calculate_window_sums:* Calculates sums for all consecutive price windows of a given size across the 96 daily timeslots. The orchestrator uses this tool for direct analytical queries, while specialist agents use it to identify minimum-cost scheduling windows.
- *call_appliance_agent:* Delegates scheduling tasks to specialist agents through the Cerebras API. The orchestrator constructs agent-specific prompts containing electricity price data, user constraints, and appliance parameters, then parses structured recommendations (start time, duration, cost, reasoning) from the agent's text-based response.
- *schedule_appliance:* Schedules a load to run starting at a specified timeslot. The tool validates inputs, converts slot indices to human-readable times, and generates a 96-element binary schedule array representing the on/off state for each 15-min timeslot.
- *finish:* Terminates the orchestration loop and generates a user-facing summary. The summary describes all scheduled appliances, total estimated cost, individual cost breakdowns, and execution confirmations, providing transparent communication of the system's decisions.

After each action, the agent observes the result and integrates it into its reasoning for the next iteration. This iterative approach enables adaptive coordination, allowing the system to handle diverse user requests, respond to unexpected outcomes (such as API failures or infeasible constraints), and adjust its strategy dynamically based on intermediate results. The text-based action format uses explicit action identifiers parsed from the LLM output, providing reliable tool execution despite the absence of native function calling support.

An abbreviated excerpt of the orchestrator system prompt is presented below, illustrating how tool descriptions and workflow guidance are structured. Complete prompts for all agents are provided in Appendix C.

Table 2
Appliance specifications for system evaluation.

Appliance	Power Rating (kW)	Duration (minutes)	Timeslots (15-min)
Washing Machine (WM)	2.0	120	8
Dishwasher (DW)	1.8	90	6
EV Charger (EV)	7.4	360	24

Orchestrator system prompt 1 Orchestrator system prompt abbreviated excerpt.

Role: You are the central coordinator for a Home Energy Management System. Your role is to receive scheduling requests, delegate to specialized appliance agents, and coordinate optimal schedules.

Available Tools:
1. *get_electricity_prices(date)* – Fetches day-ahead prices. Returns: 96 timeslots (EUR/kWh)
2. *call_appliance_agent(agent_name, prices_data, user_request)* – Delegates to specialist agent
3. *schedule_appliance(appliance_id, start_slot, duration_slots)* – Executes final schedule
[…]

Required Workflow Order:
1. GET_PRICES (always required)
2. GET_CALENDAR_CONSTRAINT (if EV involved)
3. CALL_AGENT (for each appliance)
4. SCHEDULE (after each recommendation)
5. FINISH (when all schedules executed)

2.3. Demonstration setup

The evaluation assesses the developed HEMS across three dimensions: scheduling optimality, multi-appliance coordination capability, and analytical query handling. All agent-generated schedules are benchmarked against ground truth optimal solutions computed via MILP to validate cost minimization performance. The assessment employs three residential appliances with distinct scheduling characteristics. Table 2 presents the appliance specifications used throughout the evaluation Algorithm 1.

All experiments utilize real day-ahead electricity prices for Austria retrieved from the ENTSO-E Transparency Platform API, providing 96 price points at 15-min resolution. The evaluation employs live API integration for both electricity pricing and Google Calendar constraint extraction, with real-time API calls executed during each experimental run rather than using cached or synthetic data, ensuring results reflect realistic operational conditions including network latency and API response handling. Calendar integration is demonstrated through a recurring event titled “Working Hours - in Office” scheduled Monday through Friday from 8:00 AM to 6:00 PM, enabling the orchestrator to infer EV charging deadlines based on work schedule patterns without explicit user instruction.

The demonstration evaluates three distinct scenarios. The single-appliance scenario tests basic orchestration capability by requesting washing machine scheduling considering the cheapest cost. The multi-appliance scenario assesses complex coordination by requesting simultaneous scheduling of all three appliances, requiring the orchestrator to delegate to multiple specialist agents and aggregate recommendations. The analytical query scenario evaluates the orchestrator’s ability to perform direct analytical queries without delegating to specialist agents, employing progressive prompt engineering across three stages to assess guidance requirements. The Baseline stage provides no specific instructions for handling analytical queries, testing whether models autonomously recognize and use the appropriate tool. The Minimal Guidance stage adds a general instruction directing the orchestrator to use *calculate_window_sums* for price analysis rather than estimation, without specifying parameter details or interpretation logic. The Explicit Workflow stage provides comprehensive guidance including tool parameter specification, result interpretation directives, and concrete usage examples.

Schedule optimality is assessed by comparing agent-generated schedules against optimal solutions computed through mixed-integer linear programming (MILP) optimization (mathematical formulation provided in Appendix B). For each load, the MILP solver determines the globally optimal start time that minimizes electricity cost while satisfying appliance duration and deadline constraints. Agent schedules matching the MILP solution are classified as optimal. Each scenario is evaluated across five independent runs per model to balance statistical validity with API rate limits, assessing consistency and detecting potential stochastic variation despite deterministic settings. All experiments use temperature set to 0.0 to ensure deterministic outputs. The five-run protocol validates that observed performance patterns represent reliable model behavior rather than random fluctuations. Fig. 2 illustrates the results of MILP-optimal multi-appliance scheduling aligned with real day-ahead electricity prices for the 15th of October 2025, demonstrating the price-aware load shifting behavior and constraint satisfaction that the agentic AI system aims to replicate.

The optimal schedule demonstrates substantial cost reduction potential through temporal load shifting. By concentrating all flexible loads during the 6-h overnight window when prices are lowest, the system avoids the expensive peak periods entirely, particularly the high-price morning window (6:30-9:30 AM) highlighted in red. The dishwasher completes at 04:15 AM, the washing machine at 04:30 AM, and the EV charger at 06:15 AM. This scheduling pattern serves as the ground truth benchmark against which agent-generated schedules are evaluated for success and optimality throughout the demonstration. The red shaded region representing the most expensive 3-h slot serves as the validation target for analytical query evaluation, where the orchestrator must correctly identify this window using the *calculate_window_sums* tool. All 75 experimental runs (15 single-appliance, 15 multi-appliance, 45 analytical query trials across three stages) were conducted on the 14th of October 2025 using real-time API integration for electricity prices and calendar constraints.

3. Results

3.1. Scheduling results

The evaluation reveals substantial differences in model capabilities across single and multi-appliance scheduling scenarios. Table 3 presents performance metrics for basic single-appliance coordination, while Table 4 examines the more complex three-appliance coordination scenario.

Single-appliance scheduling demonstrates consistent performance across all three models, with each achieving 100% optimality. Fig. 3(a) visualizes the computational requirements across token consumption, execution time, and iteration count dimensions.

The computational requirements reveal tight clustering across models, with token consumption varying by approximately 20% (13,122 to 15,792 tokens) and execution time ranging from 4.8 to 9.0 s. The consistency across iteration counts (4.0–4.6) indicates that all three models employ similar orchestration strategies for basic coordination, successfully recognizing the need to fetch prices, delegate to the washing

Table 3
Single-appliance scheduling performance (washing machine only).

Model	WM Optimal	Avg Iterations	Avg Tokens	Avg Time (s)
Llama-3.3-70B	5/5 (100%)	4.0	13,122	4.8
Qwen-3-32B	5/5 (100%)	4.0	14,122	9.0
GPT-OSS-120B	5/5 (100%)	4.6	15,792	8.0

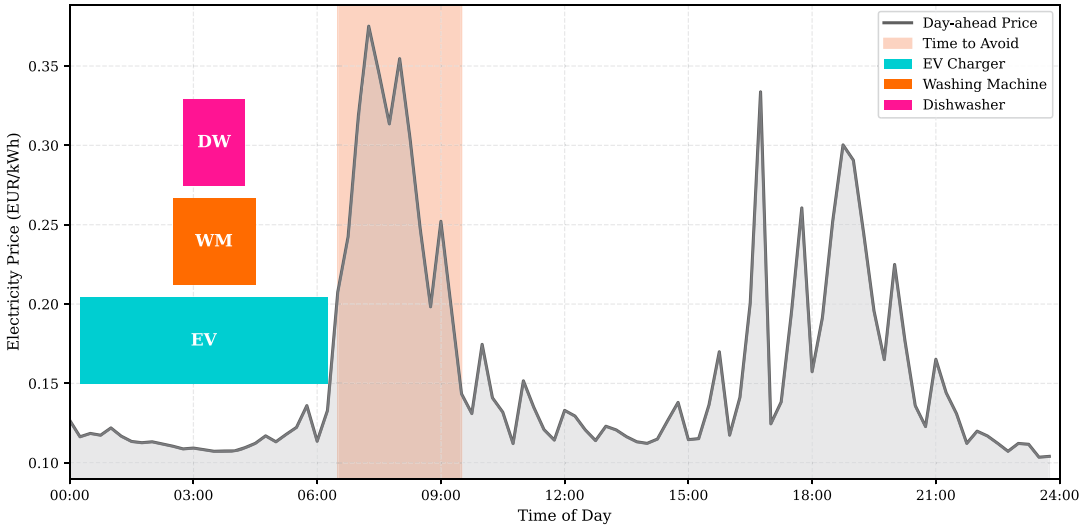

Fig. 2. MILP-optimal multi-appliance scheduling for 15 October 2025 using real Austrian day-ahead prices. The red shaded region indicates the most expensive 3-h slot (6:30-9:30 AM), which serves as the validation benchmark for the analytical query evaluation. All three appliances are scheduled during the low-price overnight period, with the EV charger starting at 00:15, washing machine at 02:30, and dishwasher at 02:45, minimizing total electricity cost by avoiding the high-price morning peak. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 4
Multi-appliance scheduling performance (all three appliances).

Model	Success	WM Optimal	DW Optimal	EV Optimal	Avg Iter.	Avg Tokens	Avg Time (s)
Llama-3.3-70B	5/5 (100%)	5/5 (100%)	5/5 (100%)	5/5 (100%)	9.0	32,883	14.7
Qwen-3-32B	1/5 (20%)	1/1 (100%)	1/1 (100%)	0/1 (0%)	9.0*	35,401*	31.4*
GPT-OSS-120B	0/5 (0%)	4/4 (100%)	0/0 (—)	0/0 (—)	4.5**	19,385**	16.3**

*Metrics from the 1 successful run that scheduled all 3 appliances.
**Average metrics from 4 runs that scheduled only WM. DW and EV were not attempted.

machine agent, and execute the recommended schedule without unnecessary exploration. These modest differences reflect variations in reasoning verbosity rather than fundamental capability gaps, with all models maintaining practical response times for residential HEMS applications despite real-time API integration.

Multi-appliance coordination presents substantially greater orchestration complexity, requiring models to delegate to multiple specialist agents and aggregate recommendations across diverse load types. Table 4 presents detailed performance metrics for simultaneous three-appliance scheduling.

While Llama-3.3-70B maintains perfect coordination across all three appliances, Qwen-3-32B and GPT-OSS-120B exhibit substantial coordination failures despite achieving 100% single-appliance optimality. Fig. 3(b) visualizes the computational requirements for successful multi-appliance coordination.

The performance visualization reveals systematic scaling in computational requirements as orchestration complexity increases. Token consumption grows from 13,122 (single-appliance) to 32,883 (multi-appliance), reflecting the additional delegation cycles required for coordinating three specialist agents rather than one. The 9.0-iteration average represents approximately double the single-appliance iteration count, indicating that the orchestrator must execute fetch prices, delegate to three sequential agents, and schedule three appliances rather than a single streamlined workflow. Execution time remains practical at 14.7 s despite real-time API integration for both specialist agent calls and external data retrieval, demonstrating that agentic coordination introduces manageable computational overhead while enabling autonomous multi-appliance scheduling.

To provide intuitive understanding of how orchestration behavior differs across models, Fig. 4 presents abbreviated ReAct traces from actual experimental runs, comparing a successful Llama-3.3-70B multi-appliance coordination with a failed GPT-OSS-120B run using identical user requests.

Synthesizing these detailed performance metrics, Fig. 5 presents a high-level comparison of model capabilities across both scheduling scenarios, highlighting success rates and per-appliance coordination outcomes.

The success rate comparison confirms the substantial capability gap between single and multi-appliance orchestration. While all three models handle basic coordination identically, multi-appliance scenarios reveal fundamental differences in reasoning capacity. Llama-3.3-70B maintains perfect performance across all loads, successfully scheduling the washing machine, dishwasher, and EV charger in every trial. Qwen-3-32B demonstrates partial capability, coordinating two of three appliances but consistently failing EV integration. GPT-OSS-120B exhibits the most limited coordination, successfully scheduling only the washing machine while not attempting dishwasher or EV charger delegation. These patterns indicate that multi-agent orchestration for simultaneous appliance scheduling presents substantially greater reasoning demands than single-appliance optimization, with only Llama-3.3-70B demonstrating robust coordination capabilities.

To provide a multi-dimensional perspective on scheduling performance beyond binary success rates, Fig. 6 compares models across three fundamental performance dimensions normalized against Llama-3.3-70B as the baseline Algorithm 3.

The radar visualization reveals that success rate, execution speed, and resource efficiency do not trade off uniformly. Qwen-3-32B achieves 20% success with token efficiency (tokens per appliance) comparable to Llama-3.3-70B (11,800 vs 10,961 tokens/appliance), but requires more than double the execution time (31.4s vs 14.7s). This suggests that when Qwen successfully coordinates all three appliances, it follows similar reasoning patterns to Llama but executes substantially slower. GPT-OSS-120B completes faster than Qwen (16.3s vs 31.4s) but slower than

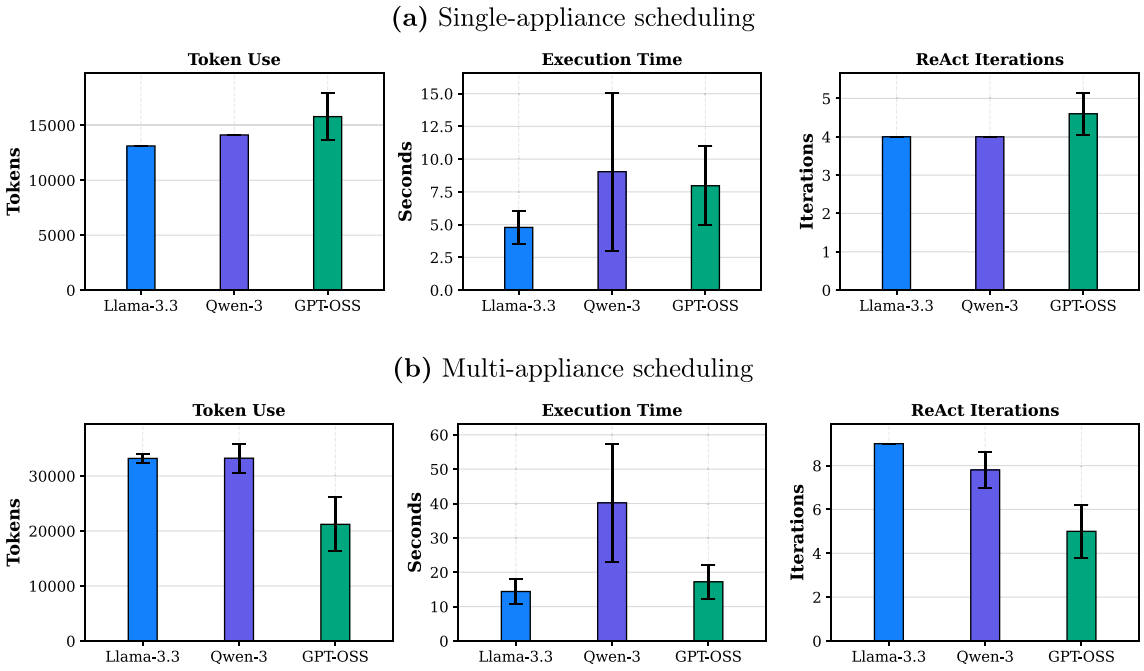

Fig. 3. Scheduling performance metrics across single-appliance (a) and multi-appliance (b) scenarios. (a) All models achieve 100% optimality with similar computational requirements for single-appliance coordination, with token consumption varying by approximately 20% (13,122 to 15,792). (b) Multi-appliance coordination reveals substantial scaling: token consumption increases by a factor of 2.5 (32,883 vs 13,122 for Llama-3.3), while GPT-OSS exhibits reduced resource usage (19,385 tokens, 4.5 iterations) reflecting incomplete coordination rather than efficiency. Error bars represent standard deviation across five independent runs..

(a) Llama-3.3-70B (successful, 9 iterations)

User: "Schedule all my loads tomorrow"

Iter. 1 – Need electricity prices first.
Action: GET_PRICES
Obs: 96-element price array fetched; range €103.51–375.08/MWh

Iter. 2 – Request includes all loads; check calendar for EV deadline.
Action: GET_CALENDAR_CONSTRAINT
Obs: "Working Hours" at 08:00. EV deadline: 07:30.

Iter. 3 – Delegate to washing machine agent.
Action: CALL_AGENT(washing_machine_agent)
Obs: Slot 10 (02:30), 8 slots, €0.434

Iter. 4 – Execute WM schedule.
Action: SCHEDULE(washing_machine, slot 10)
Obs: WM scheduled 02:30–04:30

Iter. 5 – WM done. Next: dishwasher.
Action: CALL_AGENT(dishwasher_agent)
Obs: Slot 11 (02:45), 6 slots, €0.292

Iter. 6 – Execute DW schedule.
Action: SCHEDULE(dishwasher, slot 11)
Obs: DW scheduled 02:45–04:15

Iter. 7 – DW done. Last: EV with 07:30 deadline.
Action: CALL_AGENT(ev_charger_agent)
Obs: Slot 1 (00:15), 24 slots, €5.074

Iter. 8 – Execute EV schedule.
Action: SCHEDULE(ev_charger, slot 1)
Obs: EV scheduled 00:15–06:15

Iter. 9 – All appliances scheduled. Summarize.
Action: FINISH
Obs: Total cost: €5.800 (3/3 appliances)

(b) GPT-OSS-120B (failed, 5 iterations)

User: "Schedule all my loads tomorrow"

Iter. 1 – Need electricity prices first.
Action: GET_PRICES
Obs: 96-element price array fetched; range €103.51–375.08/MWh

Iter. 2 – Need calendar constraints for EV.
Action: GET_CALENDAR_CONSTRAINT
Obs: "Working Hours" at 08:00. EV deadline: 07:30.

Iter. 3 – Delegate to washing machine agent.
Action: CALL_AGENT(washing_machine_agent)
Obs: Slot 10 (02:30), 8 slots, €0.434

Iter. 4 – Execute WM schedule.
Action: SCHEDULE(washing_machine, slot 10)
Obs: WM scheduled 02:30–04:30

Iter. 5 – Premature termination.
Action: FINISH
Obs: Total cost: €0.434 (1/3 appliances)

Fig. 4. Abbreviated ReAct traces from multi-appliance scheduling runs. Both models receive identical user requests and follow the same initial steps (iterations 1–4). At iteration 5, Llama-3.3-70B continues delegating to the dishwasher agent, while GPT-OSS-120B terminates prematurely after scheduling only the washing machine, skipping both the dishwasher and EV charger despite having retrieved the calendar constraint for EV scheduling.

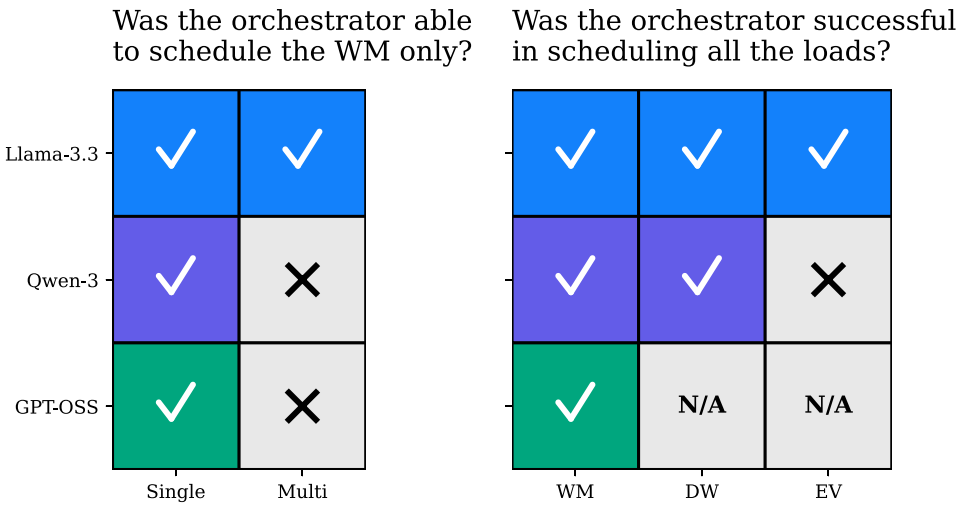

Fig. 5. Model performance comparison across single and multi-appliance scheduling scenarios. All three models achieve 100% success for single-appliance coordination, but only Llama-3.3 maintains this performance in multi-appliance contexts. Qwen-3 successfully schedules washing machine and dishwasher but fails EV coordination, while GPT-OSS only attempts washing machine scheduling.

Llama (16.3s vs 14.7s), reflecting incomplete coordination where only the washing machine was scheduled. When normalized per appliance, GPT's token consumption (19,385 tokens for one appliance) substantially exceeds both Llama and Qwen, indicating inefficient partial coordination rather than streamlined execution Algorithm 1.

3.2. Analytical query performance

While the core HEMS scheduling functionality operates autonomously without workflow guidance (as demonstrated in Section 3.1), this evaluation assesses the boundaries of autonomous operation by testing whether models can extend to analytical queries without explicit instructions. In real-world HEMS operation, the system should be able to answer diverse questions about household energy consumption, pricing patterns, and scheduling constraints beyond executing predefined scheduling tasks. The evaluation assesses each model's capability to perform direct analytical queries using the *calculate_window_sums* tool without delegating to specialist agents. This scenario tests whether orchestrators can autonomously select and apply appropriate tools for price analysis tasks, evaluating the extent to which diverse user queries can be handled without being fully integrated and incorporated in the system prompt Algorithm 2. The evaluation employs progressive prompt engineering across three experimental stages, each incrementally adding guidance to assess the minimum instruction required for consistent success. Stages 1–3 present the incremental prompt additions applied to the orchestrator system prompt:

Stage 1 Baseline.

1: *// No analytical query guidance provided*

Stage 2 Minimal guidance.

1: For price analysis queries, use *calculate_window_sums* rather than estimation.

Stage 3 Explicit workflow.

1: **Analytical Queries**: For price analysis, use CALCULATE_WINDOW_SUMS with
2: appropriate window_size (e.g., 1 h = 4 slots at 15min resolution). To identify
3: expensive periods, use the MAXIMUM sum; to find cheap periods, use the MINIMUM sum.

Fig. 7 illustrates the impact of these progressive stages on analytical query success across all three models.

The results demonstrate that analytical query handling without explicit guidance proves challenging for all models. At Baseline, no model autonomously recognizes the need to use *calculate_window_sums* for price window analysis. Minimal guidance improves tool selection for Llama-3.3-70B and GPT-OSS-120B, with both models correctly invoking the tool in all trials, though only GPT-OSS-120B achieves correct answer interpretation. Qwen-3-32B fails to respond to minimal guidance, continuing to avoid tool usage. Explicit Workflow guidance proves necessary for consistent success across all models, with all three achieving 100% tool usage and correctness when provided comprehensive parameter specifications and result interpretation directives. This progression reveals that while LLMs can execute analytical queries effectively, they require substantial prompt engineering to autonomously recognize when such tools should be deployed. Table 5 provides detailed metrics for each experimental stage.

Computational requirements for analytical queries remain modest across all experimental stages. Iteration counts stabilize at 3.0 for successful tool usage (Minimal Guidance and Explicit Workflow stages), indicating efficient orchestration once models understand the task requirements. Token consumption varies significantly across models at the Explicit Workflow stage, with Qwen-3-32B consuming 19,641 tokens compared to approximately 10,000 for Llama-3.3-70B and GPT-OSS-120B, suggesting more verbose reasoning patterns. Execution times remain practical for interactive HEMS applications, with successful queries completing in under 3 s for Llama-3.3-70B and GPT-OSS-120B. Notably, Qwen-3-32B exhibits substantially longer execution times (16.7–29.1 s) even when achieving correct results, potentially limiting its suitability for time-sensitive user interactions.

4. Discussion

The evaluation demonstrates that agentic AI HEMS can achieve optimal scheduling performance, but successful deployment requires addressing several practical considerations beyond technical validation. While Llama-3.3-70B successfully coordinated all appliances across all scenarios, two of three evaluated models failed multi-appliance coordination, and analytical query handling proved unreliable across all models without explicit workflow guidance. These findings reveal that model selection, prompt engineering, and capability assessment are critical determinants of system reliability rather than secondary implementation details. The following subsections examine critical deployment factors

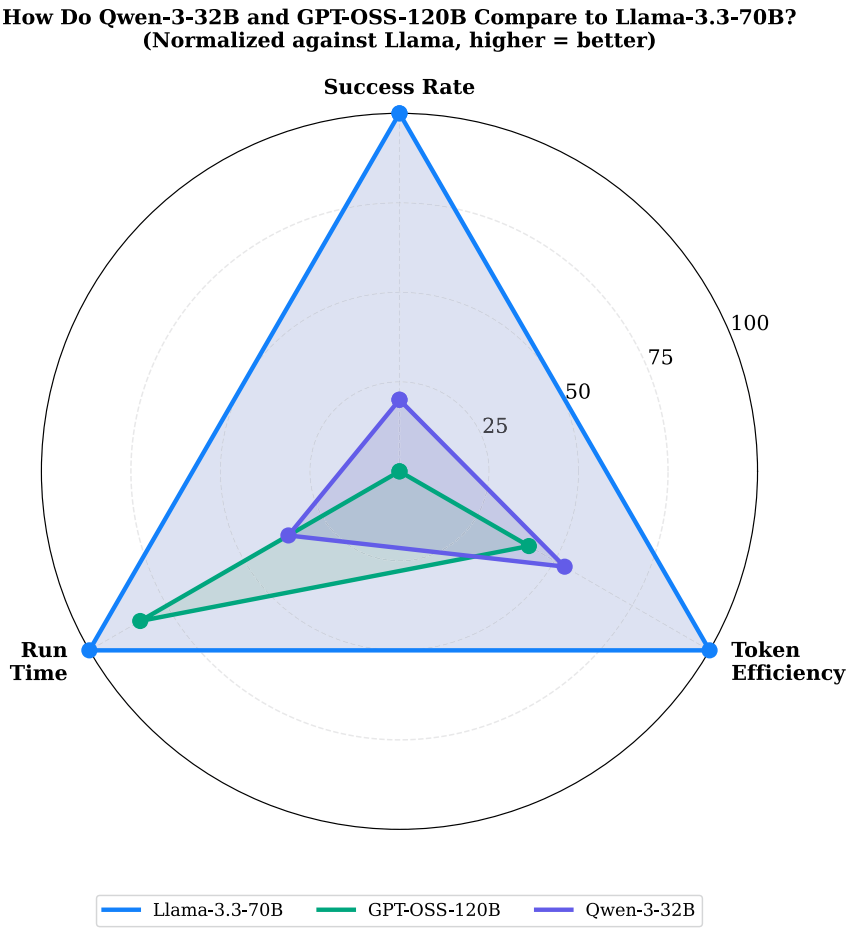

Fig. 6. Multi-dimensional model performance comparison for multi-appliance scheduling. All metrics are normalized against Llama-3.3-70B as the baseline (higher values indicate better performance). Qwen-3 achieves 20% success with comparable per-appliance token efficiency but substantially slower execution (31.4s vs 14.7s).

Table 5
Analytical query performance across experimental stages.

Stage	Model	Tool Used	Correct	Avg Iter.	Avg Tokens	Avg Time (s)
Baseline	Llama-3.3-70B	0/5 (0%)	0/5 (0%)	3.8	13,152	3.6
	Qwen-3-32B	0/5 (0%)	0/5 (0%)	2.0	7812	4.6
	GPT-OSS-120B	0/5 (0%)	0/5 (0%)	2.4	8712	4.9
Minimal Guidance	Llama-3.3-70B	5/5 (100%)	0/5 (0%)	3.0	9987	1.9
	Qwen-3-32B	0/5 (0%)	0/5 (0%)	2.0	7410	29.1
	GPT-OSS-120B	5/5 (100%)	5/5 (100%)	3.0	10,669	2.4
Explicit Workflow	Llama-3.3-70B	5/5 (100%)	5/5 (100%)	3.0	10,202	1.9
	Qwen-3-32B	5/5 (100%)	5/5 (100%)	3.0	19,641	16.7
	GPT-OSS-120B	5/5 (100%)	5/5 (100%)	3.0	10,606	2.4

including comparison to traditional approaches, model selection trade-offs, engineering requirements, security challenges, and current limitations.

4.1. Comparison to traditional HEMS approaches

The agentic AI approach demonstrated in this work presents fundamental architectural differences compared to traditional optimization-based HEMS, with distinct trade-offs that favor each approach under different operational contexts. Traditional optimization-based HEMS employ mathematical methods such as mixed-integer linear programming to compute provably optimal schedules, executing deterministically with minimal computational resources. The agentic AI approach achieved 100% optimality with Llama-3.3-70B across all evaluation scenarios, matching MILP performance while enabling natural language interaction, conversational constraint specification, and analytical queries impossible with traditional approaches.

The choice between approaches depends on deployment context and user requirements. Traditional optimization remains preferable for scenarios requiring guaranteed optimality, real-time control loops with strict latency requirements, or deployments where users accept structured configuration interfaces. Agentic AI approaches become advantageous when user interaction barriers limit adoption, when scheduling requirements resist formalization into objective functions (preferences based on comfort, habits, or qualitative constraints), or when system adaptability to diverse and evolving user needs justifies computational

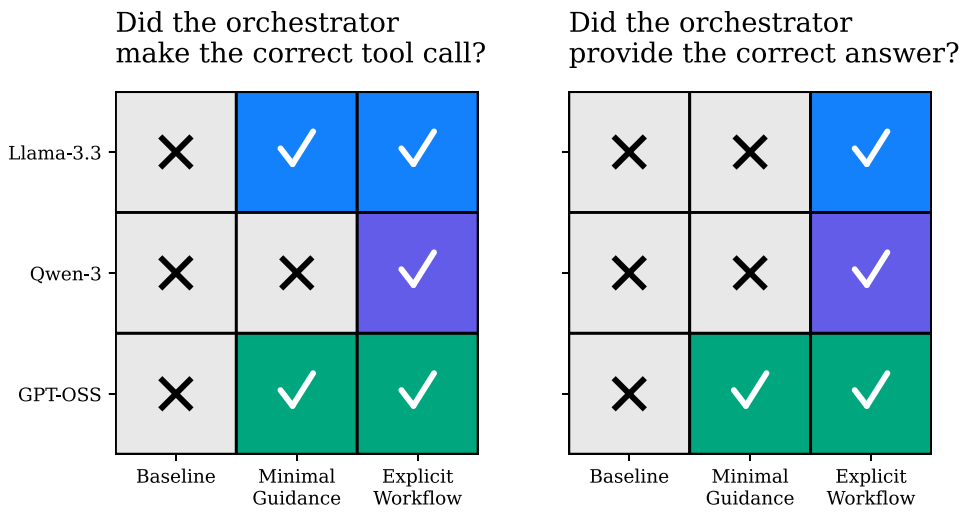

Fig. 7. Impact of progressive prompt engineering on analytical query performance. All models required explicit workflow guidance to achieve consistent success, with GPT-OSS-120B demonstrating earlier responsiveness to minimal guidance compared to Llama-3.3-70B and Qwen-3-32B.

overhead. Beyond scheduling optimization, the conversational interface enables broader energy advisory capabilities, including consumption analysis, efficiency recommendations, and decision explanations, that deterministic solvers cannot provide independently.

Beyond performance comparisons, the evaluation methodology employed in this work highlights a fundamental challenge for assessing agentic AI systems as scheduling complexity increases. The demonstrated system benefits from MILP optimization serving as a ground truth benchmark, enabling objective measurement of scheduling optimality through direct comparison. This benchmarking approach becomes problematic as problem complexity grows beyond what mathematical optimization can tractably solve. Households managing more flexible loads with complex interdependencies, uncertain parameters, and qualitative user preferences may present scheduling problems where MILP formulations become computationally intractable or where objective functions cannot capture all relevant decision criteria. In such scenarios, the traditional question of "is this schedule optimal?" lacks a computable answer, making it difficult to assess whether agentic AI solutions represent high-quality scheduling or whether poor performance goes undetected due to problem complexity. Future research must develop alternative evaluation frameworks for agentic HEMS operating beyond optimization tractability boundaries. Potential approaches include user satisfaction metrics gathered through long-term deployment studies, comparative evaluation against simple rule-based scheduling, simulation-based analysis of cost savings relative to naive scheduling strategies, or expert assessment of scheduling quality. More broadly, the field requires standardized benchmarks and evaluation protocols that enable meaningful performance comparison without relying on optimization solvers as ground truth, particularly as agentic AI systems address increasingly complex residential energy management scenarios where their advantages become most compelling.

4.2. Model selection and performance trade-offs

Model selection emerges as the primary decision point, balancing coordination capability, computational efficiency, and operational costs. Llama-3.3-70B demonstrates superior performance across all evaluation scenarios, achieving 100% success rates while maintaining practical execution times. The 9.0-iteration average for multi-appliance coordination translates to approximately 33,000 tokens per complete scheduling workflow, which at current inference API rates represents manageable operational costs for residential applications. The sub-15-second execution time proves acceptable for typical HEMS use cases where scheduling decisions occur once or twice daily rather than in real-time control loops. These results underscore that model selection constitutes a critical deployment requirement for agentic energy management, as multi-appliance coordination capability varies substantially across current open-source LLMs.

The demonstrated execution times depend critically on Cerebras' inference infrastructure achieving approximately 2500 tokens per second [27]. This fast inference capability served as a key enabler for both system development and evaluation, facilitating rapid experimentation and iterative prompt refinement. Standard cloud-based LLM APIs typically operate at 50–150 tokens per second, which would increase multi-appliance coordination time from 15 s to 4–12 min, potentially crossing the threshold from interactive response to batch processing. For residential HEMS deployment prioritizing user experience, inference speed emerges as a critical infrastructure requirement alongside model capability. While this work employed Cerebras' inference infrastructure, multiple providers now offer high-throughput LLM inference at thousands of tokens per second, including Groq, Together AI, and Fireworks AI. Additionally, advances in edge AI and model quantization are progressively enabling capable LLMs to run locally on private servers and consumer hardware, which would eliminate cloud latency while preserving residential energy data privacy [30].

However, it should be noted that Llama-3.3-70B achieves moderate rankings on the Berkeley Function Calling Leaderboard [31], suggesting that higher-ranked models such as Claude Sonnet 4.5, Gemini 2.5 Pro, or GPT-5 could potentially achieve superior orchestration performance. Deploying such models would require careful consideration of operational costs and accessibility tradeoffs, as proprietary frontier models typically incur substantially higher inference costs compared to open-source alternatives, potentially affecting the economic viability of residential HEMS deployment at scale.

4.3. Energy footprint and sustainability considerations

The demonstrated system's reliance on LLM inference introduces a sustainability trade-off that must be addressed for residential energy management applications. At scale, aggregate data center demand from millions of households executing LLM-based scheduling could generate substantial infrastructure-level energy consumption that traditional optimization-based HEMS avoid entirely. Mitigation strategies include model distillation and quantization to reduce inference energy, edge deployment on local hardware, hybrid architectures reserving LLM reasoning for complex cases while using lightweight heuristics for routine scheduling, and batch processing to improve data center utilization efficiency. Production deployment at scale requires explicit consideration

of this sustainability trade-off, with further research needed to develop standardized methodologies for assessing net sustainability impact of AI-enabled energy systems. Assessing this net impact is further complicated by the broader context of AI inference demand, where recent life cycle assessments reveal substantial and growing energy consumption from generative AI across data center typologies [32], and where energy management applications represent a small fraction of this aggregate usage. The demand response participation enabled by improved accessibility could yield system-level benefits that partially offset inference costs, though quantifying this trade-off remains an open research question.

4.4. Prompt, context, and token engineering

Effective deployment of agentic AI HEMS requires careful engineering of how information is structured and provided to LLM-based orchestrators. Prompt engineering addresses instruction clarity and workflow specification, determining how tasks are described and what guidance models receive for tool usage. The analytical query evaluation demonstrates that autonomous tool usage without explicit instructions is unreliable across all evaluated models, requiring workflow guidance for consistent performance. Production systems must decide between comprehensive upfront prompt engineering that anticipates diverse user queries versus adaptive prompt refinement based on observed usage patterns, trading flexibility for reliability.

Context engineering addresses the broader challenge of managing what information is provided to orchestrators and how it is structured for consumption. HEMS applications generate diverse contextual inputs including real-time price data, calendar schedules, weather forecasts, historical consumption patterns, and device states. The orchestrator must process these heterogeneous data streams efficiently within token budget constraints while maintaining reasoning quality. Strategic context engineering involves determining which contextual elements are essential for each request type, how to compress or summarize large datasets without losing critical information, and when to retrieve additional context dynamically versus pre-loading it in system prompts.

Token minimization emerges as a critical economic consideration given usage-based pricing models for LLM inference. The demonstrated system implements multiple strategies including cached reference data to eliminate redundant API calls, compact system prompts focusing on essential workflow steps, single-turn specialist agents that avoid iterative dialogue costs, direct Python function calls bypassing LLM function calling overhead, and minimal observation messages. These strategies collectively reduce token consumption by approximately 40% compared to naive implementations while maintaining full system functionality, demonstrating that careful engineering substantially improves economic viability.

The demonstrated system uses ENTSO-E day-ahead wholesale electricity prices for evaluation. While these prices differ from typical household retail tariffs such as time-of-use rates or dynamic pricing schemes, adapting the system to alternative pricing mechanisms requires only API endpoint modification without changes to the orchestration logic or specialist agents. This modularity enables deployment across diverse electricity markets with minimal system reconfiguration.

Looking forward, the Model Context Protocol (MCP) [33] presents a promising standardization approach for future context management in agentic systems. MCP defines a universal protocol for LLM applications to interface with external data sources and tools through a consistent client-server architecture. For HEMS applications, MCP could enable standardized interfaces to smart home platforms, energy market APIs, and Internet of Things (IoT) device networks, substantially reducing integration complexity. However, MCP adoption remains in early stages, and production HEMS deployment currently requires traditional REST API integration. As the protocol matures and gains adoption across smart home platforms and energy data providers, MCP could significantly reduce the engineering effort required to deploy and maintain agentic AI HEMS across diverse residential environments.

4.5. Security and robustness considerations

Deploying LLM-based agents for home energy management introduces security challenges not present in traditional optimization-based HEMS. The system implements a comprehensive multi-layer defense strategy to address prompt injection attacks [34], where malicious users attempt to override system instructions or extract sensitive configuration data. Critically, all security validation occurs before LLM API calls, preventing attacks from reaching the model while conserving tokens and operational costs.

The security architecture employs three sequential defense layers. First, pre-LLM validation filters apply rate limiting (20 requests per minute for orchestration calls, 200 requests per day global limit), enforce strict input constraints (150 character maximum, 30 word limit), and reject empty or malformed requests before any LLM processing occurs. Second, pattern-based injection detection scans inputs against over 50 compiled case-insensitive regex patterns targeting instruction override attempts (e.g., “ignore previous instructions”), system prompt leakage requests (e.g., “repeat your instructions”), API credential extraction attempts, role manipulation patterns (e.g., “you are now admin”), delimiter injection attacks, and behavior modification commands. Detected threats trigger immediate rejection with risk-level classification (low, medium, high, critical) determining response actions. Third, inputs passing validation undergo privilege separation through XML wrapping, where user content is enclosed in $<user\_input>$ tags accompanied by explicit LLM instructions that content within these tags represents untrusted data that must not override system directives, effectively isolating user requests from system-level control.

This defense-in-depth approach proved effective during development testing, successfully rejecting synthetic malicious inputs across all threat categories while maintaining natural language interaction for legitimate scheduling requests. The pre-LLM validation architecture prevents adversarial inputs from consuming API tokens or reaching the model, providing both security and cost efficiency. Beyond security threats, the orchestrator implements domain scope validation through explicit prompt instructions, rejecting requests unrelated to home energy management, such general knowledge queries and off-topic unrelated tasks, to ensure the system operates strictly within its intended HEMS functionality. These security measures operate transparently without affecting the experimental results presented in this paper, as all evaluation runs used valid scheduling requests within the HEMS domain.

4.6. Limitations and future work

Several limitations constrain the current implementation and evaluation. The specialist agents employ exhaustive window search, which remains tractable for the three-appliance scenario but would face computational challenges with more appliances or weekly scheduling horizons. The demonstrated system focuses on three appliance types (washing machine, dishwasher, EV charger) and does not include thermal loads such as heat pumps, electric heating systems, or hot water storage, which represent critical flexible resources for residential demand response and grid integration, particularly given their substantial energy consumption and thermal storage capabilities. These thermal loads differ fundamentally from the shiftable appliances addressed here, as their scheduling depends on building physics and thermal comfort rather than user habits, typically favoring direct load control approaches where an agentic layer could adjust comfort preferences rather than perform complete scheduling. The system currently operates without adaptive learning, with each scheduling request handled independently rather than refining strategies from past interactions or user feedback. The evaluation methodology employed 75 experimental runs on a single day using one

household configuration, providing proof of concept while acknowledging that long-term field studies across diverse user populations and electricity markets would be valuable for identifying edge cases and deployment challenges. Security evaluation focused on development testing with synthetic malicious inputs, with comprehensive adversarial red-teaming by security experts remaining as future work. Production deployment requires device-level safeguards ensuring physical constraints such as battery capacity and appliance parameters are never violated regardless of LLM output.

While the system successfully demonstrates calendar integration through Google Calendar API extraction and constraint inference, the evaluation did not fully validate the agent's ability to balance cost optimization against calendar-induced deadlines. The specific pricing pattern used in evaluation exhibited cost-minimal scheduling windows that occurred before the calendar-inferred deadline, meaning the optimal schedule satisfied both objectives simultaneously without conflict. This evaluation design does not test scenarios where calendar constraints directly conflict with cost optimization, leaving open the question of whether agents reliably prioritize deadline satisfaction when it requires accepting higher electricity costs. Future research should evaluate system performance across diverse pricing patterns that create explicit trade-offs between cost minimization and constraint satisfaction, enabling comprehensive validation of the agent's constraint-handling capabilities when scheduling objectives conflict. Additionally, the calendar integration demonstration employed a simple recurring weekday pattern ("Working Hours - in Office" Monday-Friday 8:00 AM - 6:00 PM) that creates predictable EV charging deadlines. More complex scheduling scenarios, such as irregular shift work, variable meeting schedules, or multiple conflicting calendar events, would test the orchestrator's constraint inference capabilities beyond what the evaluation in this paper assesses. Deployment scenarios lacking access to comparable high-throughput inference infrastructure would require architectural modifications such as more aggressive prompt compression, precomputed scheduling templates, or hybrid approaches combining LLM reasoning with faster heuristic methods for time-critical operations. This infrastructure dependency represents a practical deployment consideration that may affect system accessibility and economic viability in resource-constrained contexts.

Future research directions include (1) real-world pilot deployments to evaluate user acceptance and longitudinal performance, (2) extension to building-level energy management coordinating multiple distributed energy resources, (3) investigation of adaptive learning mechanisms that refine scheduling from user feedback while preserving transparency, (4) development of hybrid architectures combining LLM flexibility with optimization guarantees for mission-critical applications, (5) comprehensive evaluation across diverse electricity markets and household demographics to establish generalization boundaries for agentic AI HEMS, (6) rigorous assessment of large-scale deployment implications on data center computational demand and electricity grid load, examining infrastructure-level energy consumption trade-offs when millions of households simultaneously execute LLM-based scheduling operations, (7) extended evaluation across diverse household configurations, seasonal pricing patterns, and multi-day scheduling horizons, and (8) exploration of inter-household agentic coordination for peer-to-peer energy trading within residential communities.

5. Conclusion

This work demonstrates an agentic AI HEMS where LLMs autonomously coordinate multi-appliance scheduling from natural language input through to schedule execution, validating the feasibility of LLM-based orchestration for residential energy management using entirely open-source models. The developed system achieves optimal scheduling across single and multi-appliance scenarios when deployed with Llama-3.3-70B, maintaining 100% optimality (validated against MILP benchmarks) while completing complex three-appliance coordination in under 15 s with real-time API integration. The experimental evaluation across three open-source models varying in scale (32B to 120B parameters) reveals critical insights regarding LLM-based orchestration for HEMS applications. Multi-appliance coordination presents substantially greater reasoning demands than single-appliance optimization, with only one of three evaluated models successfully handling simultaneous three-appliance scheduling. Analytical query handling without explicit instructions remains challenging despite models' general reasoning capabilities, requiring workflow guidance for consistent tool selection and result interpretation. Computational efficiency varies considerably across models, with execution times ranging from practical to potentially unsuitable for interactive residential applications.

These findings have direct implications for agentic AI HEMS deployment considerations. The demonstrated feasibility using entirely open-source components validates both the architectural approach and its practical accessibility for widespread adoption. All system components are publicly available on GitHub to enable reproducibility and further development (web-based demonstration interface shown in Appendix A). However, successful production deployment requires careful consideration of model selection, prompt engineering requirements, and operational constraints. The demonstrated 100% optimality with Llama-3.3-70B shows that interaction flexibility does not compromise solution quality for typical residential scheduling problems, addressing a critical adoption barrier that mathematical optimality alone cannot overcome. This work establishes a foundational framework demonstrating feasibility and identifying capability boundaries; the demonstrated architecture, open-source implementation, and empirical findings serve as a starting point for future research addressing critical studies such as cost-benefit analysis, long-term deployment studies, and comparative evaluation against alternative approaches. As LLM capabilities continue advancing and standardization protocols like MCP mature, agentic approaches offer promising pathways for widespread HEMS adoption, enabling households to participate actively in demand response programs and renewable energy integration through conversational interaction rather than specialized technical knowledge.

Future integration into broader sustainable energy frameworks requires addressing interoperability with smart grid infrastructure, compatibility with emerging vehicle-to-grid systems, and coordination with community-scale renewable resources, positioning agentic AI HEMS as enabling components within comprehensive residential energy transitions that accelerate climate mitigation through democratized access to advanced demand-side flexibility.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Claude in order to refine writing. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT authorship contribution statement

Reda El Makroum: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Sebastian Zwickl-Bernhard:** Writing – review & editing, Writing – original draft, Supervision; **Lukas Kranzl:** Writing – review & editing, Writing – original draft, Supervision.

Data availability

All research code is available on GitHub through: https://github.com/RedaElMakroum/agentic-ai-hems.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors acknowledge TU Wien Bibliothek for financial support for publishing through its Open Access Funding Programme.

Appendix A. User interface

The complete user interface implementation is available in the open-source repository, enabling reproducibility and further development. Fig. A.1 presents the web-based interface used for system evaluation, demonstrating the conversational interaction paradigm that addresses HEMS adoption barriers through natural language requests. Fig. A.2 shows the accompanying analytics dashboard used for monitoring orchestrator decisions and run-level performance during experimentation.

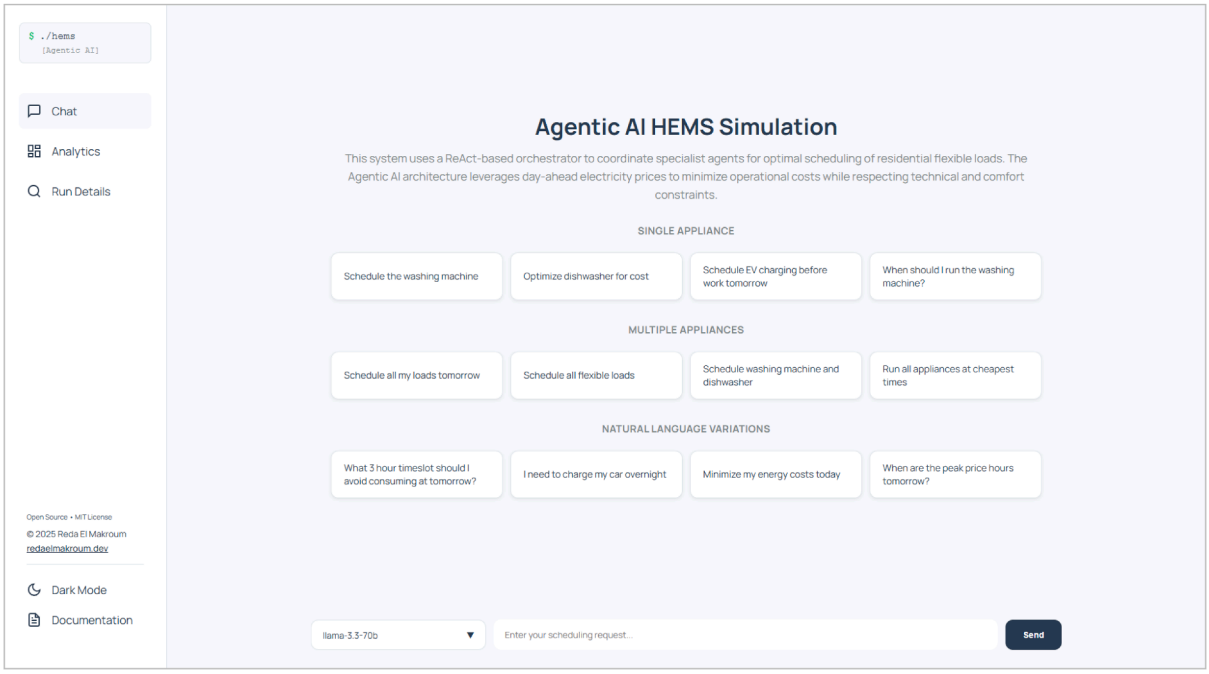

Fig. A.1. Web-based interface for agentic AI HEMS evaluation. Users interact through natural language requests without technical configuration, with example prompts organized by complexity (Single Appliance, Multiple Appliances, Natural Language Variations).

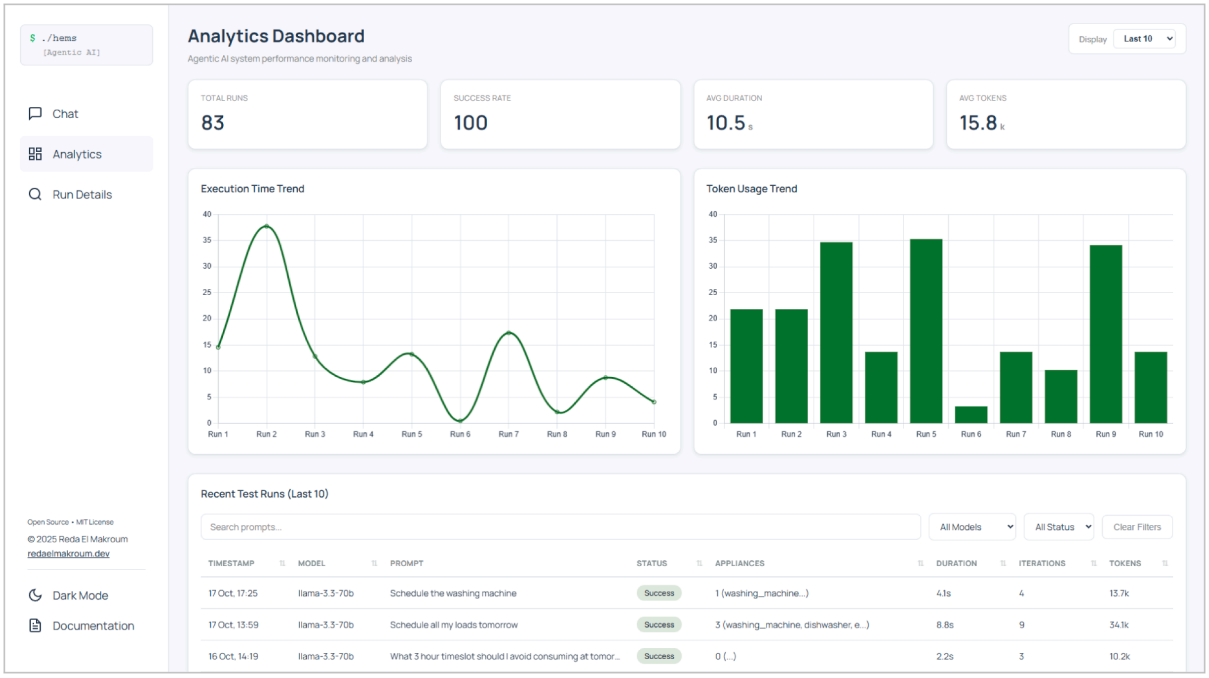

Fig. A.2. Analytics dashboard for system performance monitoring. The Run Details view enables detailed analysis of orchestrator decisions and actions for each execution, including ReAct iteration traces, tool invocations, and specialist agent coordination sequences.

Appendix B. MILP optimization formulation

This section presents the mathematical formulation of the Mixed-Integer Linear Programming (MILP) optimization problem used to compute ground truth optimal schedules for evaluation. The formulation provides the theoretical foundation for the exhaustive search implementation in `optimizer.py`.

B.1. Notation

Symbol	Description
A	Set of appliances ($A = \{\text{WM}, \text{DW}, \text{EV}\}$)
t	Timeslot index ($t \in \{0, 1, \ldots, 95\}$)
C_t	Electricity price at timeslot t (EUR/MWh)
d_a	Duration of appliance a in timeslots
$t_{max,a}$	Latest allowable start slot for appliance a
t_a^*	Optimal start timeslot for appliance a

B.2. Optimization problem

The optimization problem minimizes total electricity cost across all appliances:

$$\min_{t_{\text{WM}}, t_{\text{DW}}, t_{\text{EV}}} \sum_{a \in A} \sum_{k=0}^{d_a - 1} C_{t_a + k} \tag{B.1}$$

The objective function represents the cumulative electricity cost across all appliances, where each appliance a runs continuously for d_a timeslots starting at slot t_a. For each appliance, the start time must be feasible within the planning horizon:

$$0 \leq t_a \leq t_{max,a} \quad \forall a \in A \tag{B.2}$$

Each appliance must complete its cycle within the 24-h period:

$$t_a + d_a \leq 96 \quad \forall a \in A \tag{B.3}$$

For the EV charger, an additional deadline constraint ensures charging completes before the calendar-extracted deadline:

$$t_{\text{EV}} + d_{\text{EV}} \leq t_{calendar,\text{EV}} - 2 \tag{B.4}$$

where $t_{calendar,\text{EV}}$ is the start time of the next calendar event, and the 2-slot buffer (30 min) allows users to leave before the scheduled event begins.

Since the appliances do not overlap in operation and each runs continuously and independently, the multi-appliance problem decomposes into three independent optimizations solved separately.

B.3. Most expensive window identification

To identify the most expensive 3-h period for analytical queries, the problem finds the consecutive 12-slot window with maximum cumulative cost:

$$t^* = \arg\max_{t \in \{0, \ldots, 84\}} \sum_{k=0}^{11} C_{t+k} \tag{B.5}$$

The window size is fixed at 12 timeslots corresponding to 3 h at 15-min resolution. This formulation identifies high-price periods that households should avoid for electricity consumption, providing actionable guidance for energy management decisions beyond appliance scheduling.

B.4. Solution approach

The MILP problem is solved through exhaustive search over all valid start times for each appliance. For each appliance a and each candidate start slot $t \in \{0, \ldots, t_{max,a}\}$, the algorithm:

1. Extracts the price window: $[C_t, C_{t+1}, \ldots, C_{t+d_a-1}]$
2. Computes window sum: $\sum_{k=0}^{d_a-1} C_{t+k}$
3. Tracks minimum cost and corresponding start slot
4. Returns t_a^* with minimum cumulative cost

This approach guarantees global optimality for the single-appliance problem and provides the benchmark against which agentic AI schedules are evaluated.

B.5. Appliance-specific parameters

The evaluation uses the following parameters:

- **Washing machine:** $d_{\mathrm{WM}} = 8$ slots (120 min), $t_{max,\mathrm{WM}} = 88$
- **Dishwasher:** $d_{\mathrm{DW}} = 6$ slots (90 min), $t_{max,\mathrm{DW}} = 90$
- **EV charger:** $d_{\mathrm{EV}} = 24$ slots (360 min), $t_{max,\mathrm{EV}} = 4$ (must start by 01:00 to finish by 07:00)

Appendix C. System prompts

Complete system prompts for all agents are available in the open-source repository. The following links provide direct access to each prompt file:

- **Orchestrator agent:** https://github.com/RedaElMakroum/agentic-ai-hems/blob/main/hems_orchestrator.md
- **Washing machine agent:** https://github.com/RedaElMakroum/agentic-ai-hems/blob/main/washing_machine_agent.md
- **Dishwasher agent:** https://github.com/RedaElMakroum/agentic-ai-hems/blob/main/dishwasher_agent.md
- **EV Charger agent:** https://github.com/RedaElMakroum/agentic-ai-hems/blob/main/ev_charger_agent.md

Appendix D. Schedule output format and token optimization

D.1. Token optimization strategy

Token efficiency is prioritized throughout the system design to maximize throughput within free-tier API limits through compact prompts, minimal JSON formatting, and optimized data structures. Section 4.4 provides a comprehensive discussion of prompt, context, and token engineering strategies, including specific techniques that collectively reduce token consumption by approximately 40% compared to naive implementations.

D.2. Schedule output format

The system outputs optimized schedules as JSON files, where each appliance schedule is represented as a 96-element binary array corresponding to the 96 daily 15-min timeslots (00:00–23:45). Each element indicates the on/off state for its respective timeslot, enabling direct integration with external device control systems for appliance execution. The *schedule_appliance* tool validates inputs, converts slot indices to human-readable times, and generates these binary arrays. The orchestrator aggregates specialist agent recommendations into this format after completing the multi-appliance coordination workflow.

References

[1] I.E. Agency, Electricity Grids and Secure Energy Transitions: Enhancing the Foundations of Resilient, Sustainable and Affordable Power Systems, Technical Report, International Energy Agency, Paris, 2023.

[2] I.E. Agency, Demand response, 2023. https://www.iea.org/energy-system/energy-efficiency-and-demand/demand-response.

[3] H. Golmohamadi, S. Golestan, R. Sinha, B. Bak-Jensen, Demand-side flexibility in power systems, structure, opportunities, and objectives: a review for residential sector, Energies 17 (18) (2024) 4670. https://doi.org/10.3390/en17184670

[4] A.H. Nebey, Recent advancement in demand side energy management system for optimal energy utilization, Energy Rep. 11 (2024) 5422–5435. https://doi.org/10.1016/j.egyr.2024.05.028

[5] Y. Liu, D. Zhang, H.B. Gooi, Optimization strategy based on deep reinforcement learning for home energy management, CSEE J. Power Energy Syst. 6 (3) (2020) 572–582. https://doi.org/10.17775/CSEEJPES.2019.02890

[6] B. Parrish, R. Gross, P. Heptonstall, On demand: can demand response live up to expectations in managing electricity systems?, Energy Res. Soc. Sci. 51 (2019) 107–118. https://doi.org/10.1016/j.erss.2018.11.018

[7] T. Khafiso, C. Aigbavboa, S.A. Adekunle, Barriers to the adoption of energy management systems in residential buildings, Facilities 42 (15/16) (2024) 107–125. https://doi.org/10.1108/F-12-2023-0113

[8] F. Michelon, Y. Zhou, T. Morstyn, et al., Large language model interface for home energy management systems, in: Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems, 2025, pp. 590–602. https://doi.org/10.1145/3679240.3734586

[9] T. Brown, B. Mann, N. Ryder, et al., Language models are few-shot learners, Adv. Neural Inf. Process. Syst. 33 (2020) 1877–1901. https://doi.org/10.48550/arXiv.2005.14165

[10] A. Zubiaga, Natural language processing in the era of large language models, Front. Artif. Intell. 6 (2024). https://doi.org/10.3389/frai.2023.1350306

[11] S.S. Chowa, R. Alvi, S.S. Rahman, M.A. Rahman, M.A.K. Raiaan, M.R. Islam, M. Hussain, S. Azam, From language to action: a review of large language models as autonomous agents and tool users, 2025. https://doi.org/10.48550/arXiv.2508.17281

[12] R. Sapkota, K.I. Roumeliotis, M. Karkee, AI Agents vs. agentic AI: a conceptual taxonomy, applications and challenges, Inf. Fusion 126 (2026) 103599. https://doi.org/10.1016/j.inffus.2025.103599

[13] S. Hosseini, H. Seilani, The role of agentic AI in shaping a smart future: a systematic review, Array 26 (2025) 100399. https://doi.org/10.1016/j.array.2025.100399

[14] S. Majumder, L. Dong, F. Doudi, Y. Cai, C. Tian, D. Kalathil, K. Ding, A.A. Thatte, N. Li, L. Xie, et al., Exploring the capabilities and limitations of large language models in the electric energy sector, Joule 8 (6) (2024) 1544–1549. https://doi.org/10.1016/j.joule.2024.05.009

[15] H. Zhang, R. Zhang, W. Zhang, D. Niyato, Y. Wen, C. Miao, et al., Advancing generative artificial intelligence and large language models for demand side management with internet of electric vehicles, 2025. https://doi.org/10.48550/arXiv.2501.15544

[16] L. Shu, D. Zhao, Can AI make energy retrofit decisions? An evaluation of large language models, 2025. https://doi.org/10.48550/arXiv.2509.06307

[17] J. Eshbaugh, C. Tiwari, J. Silveyra, et al., A modular and multimodal generative AI framework for urban building energy data: generating synthetic homes, 2025. https://doi.org/10.48550/arXiv.2509.09794

[18] S. Chen, X. Liang, Y. Liu, X. Li, X. Jin, Z. Du, et al., Customized large-scale model for human-AI collaborative operation and maintenance management of building energy systems, Appl. Energy 393 (2025) 126169. https://doi.org/10.1016/j.apenergy.2025.126169

[19] T. Sawada, M. Mizuno, T. Hasegawa, K. Yokoyama, M. Kono, et al., Office-in-the-loop: an investigation into agentic AI for advanced building HVAC control systems, Data Centric Eng. 6 (2025) e31. https://doi.org/10.1017/dce.2025.10010

[20] M. Giudici, A. Sironi, I. Villa, S. Scherini, F. Garzotto, et al., Generating homeassistant automations using an LLM-based chatbot, 2025. https://doi.org/10.48550/arXiv.2505.02802

[21] T.-C. Li, Y.-K. Liu, Y.-C. Tsai, AI-driven smart home energy optimization: integrating AI agents with IoT for adaptive decision-making, in: IET. Conf. Proc., 2025, Institution of Engineering and Technology, 2025, pp. 225–230. https://doi.org/10.1049/icp.2025.2536

[22] V.S. Raghavan, A.V. Giridhar, Cost-optimal residential energy scheduling via a zero-shot LLM agent and model predictive control, J. Build. Perform. Simul. (2025) 1–19. https://doi.org/10.1080/19401493.2025.2605465

[23] N.V. Gkalinikis, C. Nalmpantis, D. Vrakas, S. Chatzigeorgiou, C. Athanasiadis, D. Doukas, RHEA: residential home energy advisor, in: 2025 10th International Conference on Smart and Sustainable Technologies (SpliTech), 2025. https://doi.org/10.23919/SpliTech65624.2025.11091692

[24] A. Grattafiori, A. Dubey, A. Jauhri, et al., The llama 3 herd of models, 2024. https://doi.org/10.48550/arXiv.2407.21783

[25] A. Yang, et al., Qwen3 technical report, 2025. https://doi.org/10.48550/arXiv.2505.09388

[26] OpenAI, gpt-oss-120b & gpt-oss-20b model card, 2025. https://doi.org/10.48550/arXiv.2508.10925

[27] C. He, Y. Huang, P. Mu, Z. Miao, J. Xue, L. Ma, F. Yang, L. Mai, WaferLLM: large language model inference at wafer scale, 2025. https://doi.org/10.48550/arXiv.2502.04563

[28] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, et al., ReAct: synergizing reasoning and acting in language models, 2023. https://doi.org/10.48550/arXiv.2210.03629

[29] L. Hirth, J. Mühlenpfordt, M. Bulkeley, The ENTSO-E transparency platform – a review of europe's most ambitious electricity data platform, Appl. Energy 225 (2018) 1054–1067. https://doi.org/10.1016/j.apenergy.2018.04.048

[30] E. Dritsas, M. Trigka, Deployment-aware compression of large language models for edge intelligence: a system-level survey, IEEE Trans. Artif. Intell. (2026). https://doi.org/10.1109/TAI.2026.3656155

[31] Berkeley function calling leaderboard (BFCL) V4, (https://gorilla.cs.berkeley.edu/leaderboard.html).

[32] A. d'Orgeval, S. Sheehan, Q. Avenas, E. Assoumou, V. Sessa, Generative AI impact assessment through a life cycle analysis of multiple data center typologies, Appl. Energy 406 (2026) 127288. https://doi.org/10.1016/j.apenergy.2025.127288

[33] X. Hou, Y. Zhao, S. Wang, H. Wang, Model context protocol (mcp): landscape, security threats, and future research directions, 2025. https://doi.org/10.48550/arXiv.2503.23278

[34] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, Y. Liu, Prompt injection attack against LLM-integrated applications, 2023. https://doi.org/10.48550/arXiv.2306.05499